

Trabalho Prático – Módulo 1

47224	André Graça
46052	Daniel Caseiro
48459	Diogo Santos

Orientadores	Vítor Fialho
	Artur Ferreira

Relatório realizado no âmbito de Comunicação digital,
do curso de licenciatura em Engenharia Informática e de Computadores
Semestre de Verão 2022/2023

Junho de 202

Resumo

Este relatório aborda dois conjuntos de exercícios relacionados à transmissão de dados por meio de canais com erros. No primeiro conjunto, foi analisado o desempenho de duas técnicas de correção de erros (código de repetição e código de Hamming) num Binary Symmetric Channel (BSC).

Foi utilizado um Sistema de Comunicação de Dados (SCD) em modo simplex entre um Arduino (emissor) e um PC (recetor), utilizando a plataforma Arduino e a linguagem Python com o módulo pyserial. O SCD tem como objetivo transmitir os primeiros N termos de uma progressão geométrica, com opção de realizar a detecção de erros usando a técnica de checksum de Fletcher. Resultados experimentais foram apresentados para comprovar o funcionamento correto do SCD em diferentes situações de transmissão.

Por fim, foram introduzidos padrões de erro em “burst” na sequência de símbolos recebidos pelo PC. Os resultados experimentais foram comentados e a capacidade de detecção de erros da técnica de checksum de Fletcher foi caracterizada.

O relatório aborda uma variedade de técnicas de correção e detecção de erros, fornecendo resultados experimentais e comentários sobre o desempenho de cada técnica em diferentes cenários de transmissão.

Palavras-chave: Transmissão de dados; Canais com erros; Técnicas de correção de erros; Binary Symmetric Channel (BSC); Sistema de Comunicação de Dados (SCD); Arduino; Pyserial; Modo simplex; Detecção de erros; Checksum de Fletcher

Abstract

This report addresses two sets of exercises related to data transmission through error-prone channels. In the first set, the performance of two error correction techniques (repetition code and Hamming code) was analyzed in a Binary Symmetric Channel (BSC).

A Data Communication System (DCS) was used in simplex mode between an Arduino (transmitter) and a PC (receiver), using the Arduino platform and the Python language with the pyserial module. The DCS aims to transmit the first N terms of a geometric progression, with the option to perform error detection using the Fletcher checksum technique. Experimental results were presented to demonstrate the correct functioning of the DCS in different transmission scenarios.

Finally, burst error patterns were introduced into the sequence of symbols received by the PC. The experimental results were discussed, and the error detection capability of the Fletcher checksum technique was characterized.

The report covers a variety of error correction and error detection techniques, providing experimental results and comments on the performance of each technique in different transmission scenarios.

Keywords: Data transmission; Error-prone channels; Error correction techniques; Binary Symmetric Channel (BSC); Data Communication System (DCS); Arduino; Pyserial; Simplex mode; Error detection; Fletcher checksum.

Índice

RESUMO	III
ABSTRACT	V
LISTA DE FIGURAS	VIII
1. INTRODUÇÃO	1
2. SOLUÇÃO PROPOSTA - GRANDES IDEIAS	2
2.1 ALÍNEA 1	2
2.1.1 Alínea (a) – <i>Binary Symmetric Channel (BSC)</i>	2
2.1.2 Alínea (b) – <i>Técnica de Entrelaçamento</i>	5
2.2 ALÍNEA 2 – SISTEMA SCD	6
2.2.1 Alínea (a) – <i>Correto funcionamento do SCD</i>	6
2.2.2 Alíneas (b e c) – <i>Inserção de padrões de erros</i>	7
3. AVALIAÇÃO EXPERIMENTAL	8
3.1 ALÍNEA 1(I) – SEM CONTROLO DE ERROS	8
3.2 ALÍNEA 1(II) – CÓDIGO DE REPETIÇÃO (3,1)	9
3.3 ALÍNEA 1(III) – CÓDIGO DE HAMMING (7,4)	10
3.4 ALÍNEA 1(B) – TÉCNICA DE ENTRELAÇAMENTO	11
3.5 ALÍNEA 2(A) – FUNCIONAMENTO DE SCD	12
3.6 Alíneas 2(b) e 2(c) – <i>Inserção de Padrões de erro</i>	13
4. CONCLUSÕES	14

Lista de Figuras

Figura 1 - Binary Symmetric Channel	2
Figura 2 - BSC com código repetição (3,1).....	3
Figura 3 - BSC com código de Hamming (7,4)	4
Figura 4 - Técnica de entrelaçamento	5
figura 5 - Esquema do SCD a funcionar em modo simplex	6
figura 6 - Esquema de verificação de erro por checksum	7
Figura 7 - Alice29.txt sem controlo de erros.....	8
Figura 8 – Alice29.txt com código repetição (3,1).....	9
Figura 9 - Alice29.txt com Código Hamming (7,4)	10
figura 10 - Técnica de entrelaçamento com código de Hamming (7,4)	11
figura 11 - Técnica de entrelaçamento com código de repetição (3,1)	11
figura 12 - 10 primeiros termos da progressão geométrica	12
figura 13 - Resultados de aplicações de erros sobre progressão geométrica.....	13

1. Introdução

Neste trabalho prático, vamos explorar o tema da transmissão de dados em canais de comunicação sujeitos a erros. Em particular, iremos focar em dois exercícios distintos, nos quais analisaremos o desempenho de diferentes técnicas de controle de erros.

No primeiro exercício, iremos introduzir a técnica de entrelaçamento (interleaving). Essa técnica envolve a reorganização dos dados antes da transmissão, de modo a minimizar o impacto dos erros em sequências consecutivas de bits. Serão realizadas comparações entre os resultados obtidos com a aplicação do entrelaçamento e os resultados do primeiro exercício, permitindo-nos avaliar o ganho proporcionado por essa técnica na redução dos erros de transmissão.

No segundo exercício, iremos considerar o BSC, um modelo simplificado de canal de comunicação no qual os bits transmitidos podem sofrer erros de forma independente com uma determinada probabilidade. Serão utilizados diferentes valores de probabilidade de erro de bit para avaliar o desempenho da transmissão. Através da comparação das taxas de erro de bit (BER) em diferentes cenários, aplicando ou não técnicas de correção de erros, poderemos analisar os efeitos dessas técnicas na qualidade da transmissão de dados.

Ao realizar esses exercícios e analisar os resultados, obteremos uma compreensão mais aprofundada do desempenho das diferentes estratégias de controle de erros e entrelaçamento em canais de comunicação. Essa análise nos dará uma visão mais clara sobre a importância de detecção e correção de erros, permitindo-nos avaliar as vantagens e desvantagens de cada abordagem. Com isso, seremos capazes de aprimorar a confiabilidade das transmissões de dados em ambientes sujeitos a erros, destacando a necessidade de técnicas eficazes de controle de erros na transmissão de informações.

2. Solução Proposta - Grandes Ideias

2.1 Alínea 1

2.1.1 Alínea (a) – Binary Symmetric Channel (BSC)

2.1.1.1 Alínea (i) – Sem controlo de erros

A execução do código em questão está relacionada à implementação e análise do Binary Symmetric Channel (BSC) e da taxa de erro de bit (BER - Bit Error Rate).

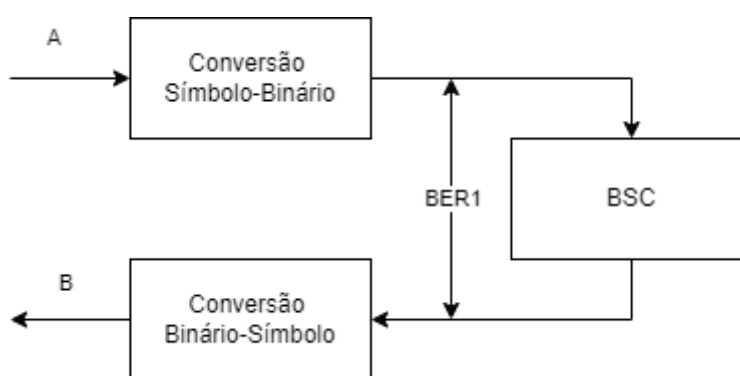


Figura 1 - Binary Symmetric Channel

No exercício proposto (a) – (i), está a ser solicitado que seja calculado o BER1 entre a entrada e a saída do BSC, sem controlo de erros. Através da análise dos valores de BER1 obtidos para cada ficheiro de teste, é possível avaliar a eficiência do BSC na introdução de erros de bit e determinar a taxa de erro de bit resultante.

Resumidamente, usando o módulo “Converters”, o conteúdo do ficheiro é convertido numa sequência binária. Tendo em consideração a existência de um array bers = [0.1, 0.01, 0.001, 0.0001, 0.00001], para cada valor p do array bers, é aplicado o BSC na sequência binária convertida e calculado o número de bits trocados (erros1) comparando a sequência original com a sequência resultante após a aplicação do BSC.

De seguida, é calculada a taxa de erro de bit (BER1) dividindo o número de bits trocados pelo comprimento da sequência resultante e é feita a conversão da sequência binária resultante para caracteres utilizando a função deconverter do módulo "Converters". Depois é calculado o número de símbolos diferentes (difChars) entre o conteúdo original do arquivo e o conteúdo resultante após a aplicação do BSC.

Por fim, são impressos os valores do BER desejado, o BER calculado, o número total de bits que passaram pelo BSC e o número de símbolos diferentes entre o conteúdo original e o conteúdo resultante.

2.1.1.2 Alínea (ii) – Código de repetição (3,1)

O código em questão realiza a aplicação de um código de repetição (3, 1) sobre o Binary Symmetric Channel (BSC) e calcula a taxa de erro de bit (BER) após essa aplicação. O código realiza uma análise do desempenho do código de repetição (3, 1) em relação à taxa de erro de bit, evidenciando a capacidade de correção de erros desse código.

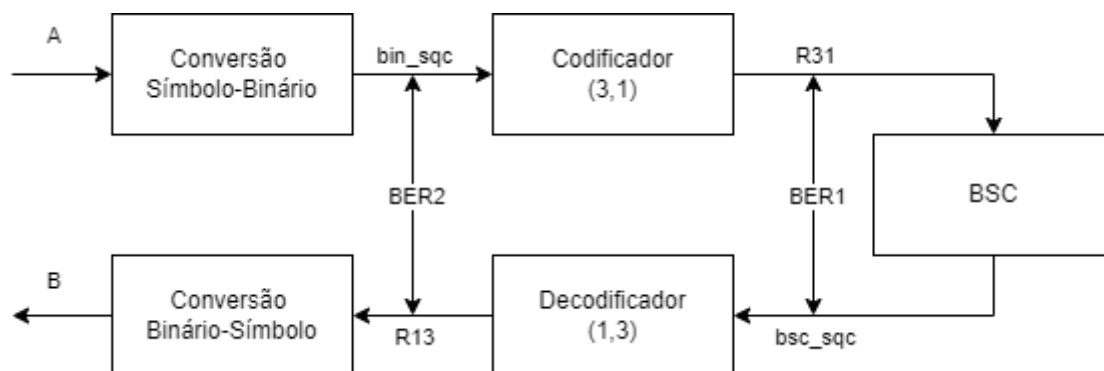


Figura 2 - BSC com código repetição (3,1)

Semelhante à alínea (i), é definido o array bers, onde depois vão ser processados os ficheiros no array files.

Na sua execução, é armazenado o ficheiro lido e, posteriormente, convertido para uma sequência binária usando o módulo “Converters”. De seguida, é aplicado o código de repetição (3,1) na sequência binária, criando uma sequência binária resultante denominada R31.

Assim sendo, para cada valor de probabilidade de error de bit (p) no array bers, é aplicado o BSC na sequência R31 com a probabilidade de erro p, criando uma sequência resultante denominada de bsc_sqc, onde vai ser calculado o número de bits trocados (erros1) comparando a sequência original R31 com a sequência resultante bsc_sqc e, por último, é calculado o BER1 dividindo o número de bits trocados pelo tamanho da sequência bsc_sqc.

De seguida, é realizada a decodificação da sequência bsc_sqc usando o código de repetição (3, 1), gerando uma sequência resultante denominada R13 e calculado o BER2 dividindo o número de bits trocados por len(bin_sqc) (comprimento da sequência binária original).

Por fim, a sequência resultante R13 é convertida novamente para caracteres utilizando a função deconverter do módulo "Converters" e são exibidos o número total de bits que passaram pelo BSC (comprimento da sequência R31), o número de símbolos diferentes entre o conteúdo original do arquivo A e o conteúdo resultante B.

2.1.1.3 Alínea (iii) – Código de Hamming (7,4)

O código apresentado realiza a aplicação do código de Hamming (7, 4) sobre o Binary Symmetric Channel (BSC) e calcula a taxa de erro de bit (BER) antes e depois da aplicação do código de Hamming.

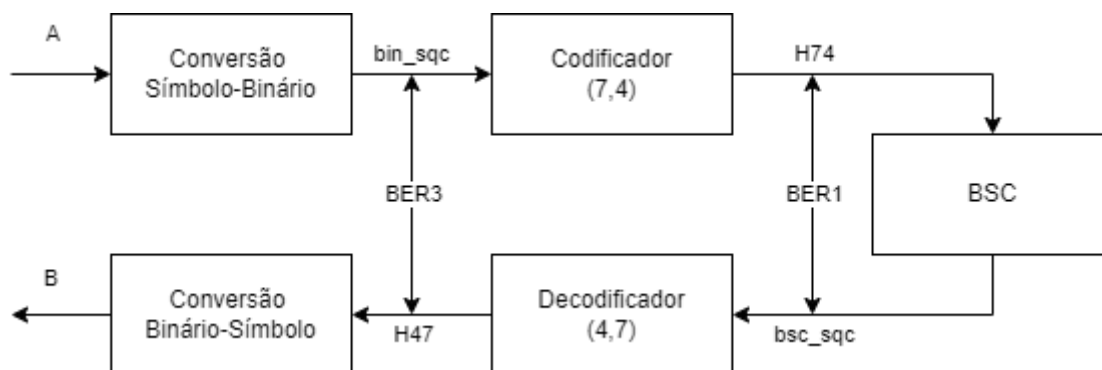


Figura 3 - BSC com código de Hamming (7,4)

Em semelhança às alíneas (i) e (ii), o ficheiro vai ser lido e convertido para binário e este valor vai ser colocado na variável A. Contudo, nesta alínea é aplicado o código Hamming (7,4) na sequência binária convertida, criando uma sequência resultante denominada H74. Assim sendo, para cada valor de probabilidade de erro de bit (p) no array bers vai ser aplicado o BSC na sequência H74 com a probabilidade de erro p, gerando uma sequência resultante denominada bsc_sqc e vai ser calculado o número de bits trocados (errors1) comparando a sequência original H74 com a sequência resultante bsc_sqc, por fim, é calculado o BER1 dividindo o número de bits trocados pelo tamanho da sequência bsc_sqc.

De seguida, é realizada a decodificação da sequência bsc_sqc usando o código de Hamming (7, 4), criando uma sequência resultante denominada H47. 1. É calculado o número de bits trocados (errors2) comparando a sequência original (sequência binária convertida) com a sequência resultante H47 e o valor de BER2 dividindo o número de bits trocados pelo comprimento da sequência binária original, bin_sqc.

Por fim, a sequência resultante H47 é convertida novamente para caracteres utilizando a função deconverter do módulo "Converters" e são exibidos o número total de bits que passaram pelo BSC (comprimento da sequência H74), o número de símbolos diferentes entre o conteúdo original do arquivo A e o conteúdo resultante B.

O código analisa a eficiência do código de Hamming (7, 4) na correção de erros introduzidos pelo BSC e avalia a taxa de erro de bit resultante.

2.1.2 Alínea (b) – Técnica de Entrelaçamento

A alínea (b) envolve a aplicação da técnica de entrelaçamento (interleaving) no processo de transmissão dos dados, utilizando o esquema representado na Figura 2. O objetivo é avaliar o desempenho da transmissão de quatro arquivos, considerando a aplicação do código de Hamming (7, 4) e comparando os resultados obtidos com a alínea anterior (alínea (a)).

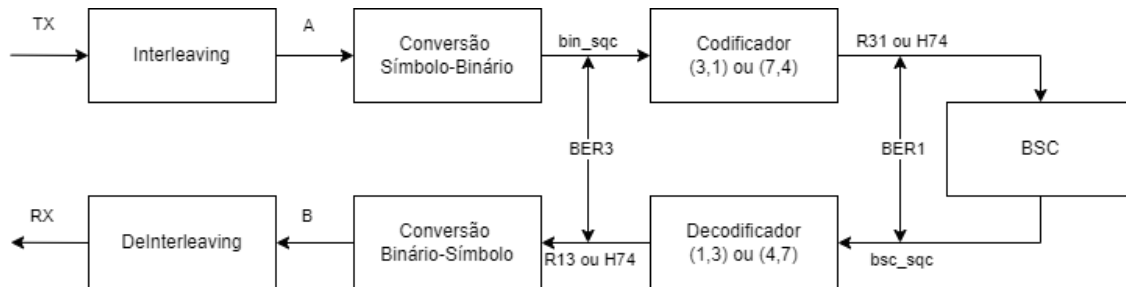


Figura 4 - Técnica de entrelaçamento

O código apresentado consiste em duas funções: `r31()` e `hamming()`, cada uma aplicando um método de correção de erros diferentes.

Na função `r31()`, são definidos os valores de probabilidade de erro de bit (`bers`) e o texto a ser transmitido (`TX`). Em seguida, é realizado o entrelaçamento do texto utilizando a função `interleaving()`. A sequência resultante é convertida numa sequência binária utilizando as funções `matrix_to_binary()` e `array_to_string()`. Posteriormente, é aplicado o código de repetição (3, 1) na sequência binária, gerando a sequência `R31`.

A seguir, para cada valor de probabilidade de erro de bit no array `bers`, é aplicado o BSC na sequência `R31` com a probabilidade de erro correspondente. O número de bits trocados (`errors1`) é calculado comparando a sequência original `R31` com a sequência resultante do BSC. O `BER1` é calculado dividindo o número de bits trocados por `len(bsc_sqc)`, onde `bsc_sqc` é a sequência resultante do BSC. Em seguida, é realizada a decodificação da sequência `bsc_sqc` utilizando a função `decod()`. O número de bits trocados (`errors2`) é calculado comparando a sequência binária original com a sequência resultante da decodificação. O `BER2` é calculado dividindo `errors2` por `len(bin_sqc)`, onde `bin_sqc` é a sequência binária original.

A sequência resultante da decodificação é convertida novamente numa matriz utilizando as funções `binary_to_matrix()` e `deinterleaving()`. É exibido o conteúdo da matriz `RX`, que representa o texto resultante da transmissão.

Por fim, são exibidos o número total de bits que passaram pelo BSC (`len(R31)`), o número de símbolos diferentes entre o texto original `TX` e o texto resultante `RX` e os valores de `BER1` e `BER2`.

A função `hamming()` segue um processo semelhante à função `r31()`, com a diferença de que, em vez de aplicar o código de repetição, é aplicado o código de Hamming (7, 4) na sequência binária após o entrelaçamento.

O objetivo dessas etapas é avaliar o desempenho da técnica de entrelaçamento em conjunto com o código de Hamming na correção de erros introduzidos pelo BSC. Comparando os resultados obtidos com os da alínea anterior (alínea (a)), é possível verificar se a utilização do entrelaçamento melhora a taxa de erro de bit e a integridade dos dados transmitidos.

2.2 Alínea 2 – Sistema SCD

Nesta alínea, foi proposto criar um Sistema de Comunicação de Dados (SCD) utilizando o Arduino como emissor e o PC como receptor, estabelecendo uma conexão via USB em modo simplex. O objetivo é transmitir os primeiros N termos de uma progressão geométrica, definida por um primeiro termo u e uma razão r , como implementado em um exercício anterior. Também está envolvida a possibilidade de realizar a detecção de erros utilizando a técnica Fletcher checksum, que é um algoritmo de verificação de integridade dos dados transmitidos. Essa detecção de erros pode ser ativada ou desativada através de um parâmetro de entrada.

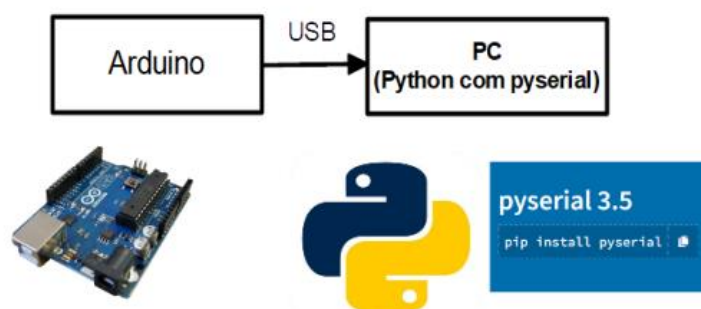


figura 5 - Esquema do SCD a funcionar em modo simplex

2.2.1 Alínea (a) – Correto funcionamento do SCD

O 1º objetivo era estabelecer a comunicação entre o Arduino e o PC, utilizando um cabo USB para a conexão física. Para isso, foi utilizado o módulo pyserial da linguagem Python, que permite a comunicação serial entre os dispositivos.

Uma vez estabelecida a comunicação, o Arduino foi configurado como emissor e o PC como receptor. O Arduino foi programado para gerar sequências de N termos de uma progressão geométrica. Esses termos foram transmitidos via comunicação serial para o PC.

No programa em Python, a biblioteca pyserial foi utilizada para receber os dados enviados pelo Arduino. A função `serial.Serial()` foi utilizada para configurar a porta serial, especificando a porta correta e a taxa de transmissão. Após esta configuração, foi utilizado o método `readline()` para ler

as linhas de dados enviadas pelo Arduino durante N vezes. Esses dados foram recebidos como sequências de caracteres e convertidos para valores numéricos, conforme necessário.

Para comprovar o bom funcionamento do emissor e do recetor, os termos recebidos foram exibidos na consola para verificar se os termos transmitidos pelo Arduino foram corretamente recebidos e processados pelo PC.

2.2.2 Alíneas (b e c) – Inserção de padrões de erros

Nesta alínea, começámos por fazer a emissão da sequência de bits da progressão geométrica normalmente, tal como na alínea anterior, mas introduzimos um erro numa posição aleatória da sequência de 32 bits na receção de cada trama fazendo a operação XOR entre o valor original e o do padrão de erro. Com isto simulámos um cenário onde ocorre um erro de transmissão (causado por ruído, por exemplo).

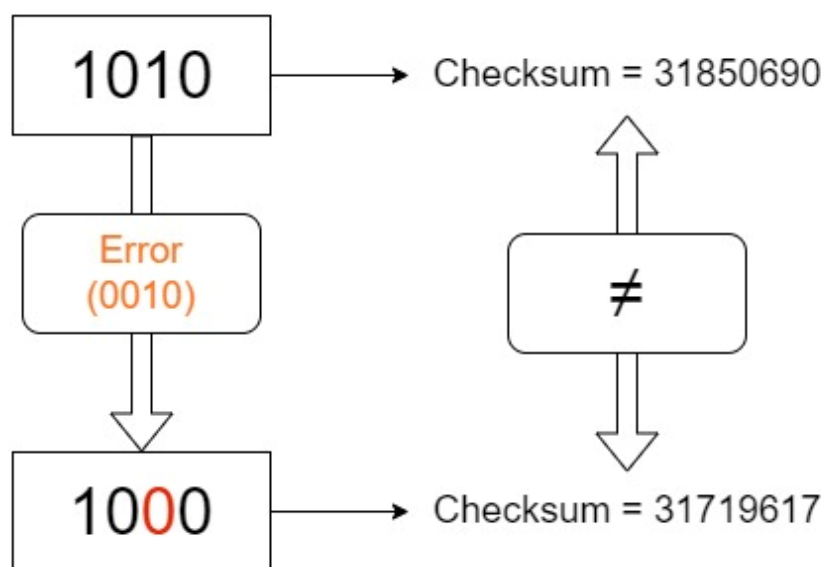


figura 6 - Esquema de verificação de erro por checksum

Em seguida, utilizámos a técnica de detecção de erros conhecida como Fletcher checksum. Para cada termo recebido, calculamos o checksum utilizando a função *getFletcher32()*, pois trabalhamos com floats, fornecida pelo docente. Comparando o checksum calculado com o checksum recebido, determinamos se ocorreu ou não um erro durante a transmissão.

3. Avaliação Experimental

Nesta secção irão ser apresentados os resultados experimentais relativos ao ficheiro *alice29.txt*, fornecido pelos docentes. O objetivo desta secção é comentar os resultados obtidos e expor observações durante a análise dos mesmos.

3.1 Alínea 1(i) – Sem controlo de erros

```
./testFiles/alice29.txt
-----
Número de caracteres:144874
Número de bits trocados:115825

Desired BER1: 0.1
Actual BER1: 0.0999359788505874
Nº total de bits que passam no BSC: 1158992
Nº de símbolos diferentes entre A e B: 82541
-----
Número de caracteres:144874
Número de bits trocados:13

Desired BER1: 1e-05
Actual BER1: 1.1216643428082333e-05
Nº total de bits que passam no BSC: 1158992
Nº de símbolos diferentes entre A e B: 13
```

Figura 7 - Alice29.txt sem controlo de erros

Pela figura 5, podemos observar que o valor obtido de BER é muito próximo do valor expetado, sendo o valor expetado de 0,1 e o valor obtido de 0,0999. No primeiro teste temos que após a passagem pelo canal BSC, o ficheiro continha 144874 caracteres, ou seja, $144874 * 8 = 1158992$ bits e que 115825 desses bits foram trocados, repare que 115825 é aproximadamente 10% de o número total de bits que passaram e que compõem o ficheiro. Podemos observar também que $\frac{\#Símbolos\ diferentes}{\#Símbolos\ total} = \frac{82541}{144874} = 0,56974 \approx 56,97\%$ da mensagem está errada para uma BER de 10%.

Para o último teste, podemos observar que o valor de BER obtido é $1,12 \cdot 10^{-5}$ que é aproximadamente $1,0 \cdot 10^{-5}$ que era o valor expetado. Neste caso como os erros acontecem com menos frequência logo com mais distância um dos outros, existe uma proporcionalidade direta de 1:1 entre os bits trocados e o nº de símbolos trocados.

3.2 Alínea 1(ii) – Código de repetição (3,1)

./testFiles/alice29.txt	
Número de caracteres:144874	Número de caracteres:144874
Número de bits trocados:347329	Número de bits trocados:314
Desired BER1: 0.1	Desired BER1: 0.0001
Actual BER1: 0.09989398833929254	Actual BER1: 9.030835990815007e-05
Desired BER2: 0	Desired BER2: 0
Actual BER2: 0.028220212046329914	Actual BER2: 0.0
Nº total de bits que passam no BSC: 3476976	Nº total de bits que passam no BSC: 3476976
Nº de símbolos diferentes entre A e B: 29639	Nº de símbolos diferentes entre A e B: 0
Número de caracteres:144874	Número de caracteres:144874
Número de bits trocados:34973	Número de bits trocados:32
Desired BER1: 0.01	Desired BER1: 1e-05
Actual BER1: 0.010058453092572396	Actual BER1: 9.203399735862427e-06
Desired BER2: 0	Desired BER2: 0
Actual BER2: 0.00027610199207587285	Actual BER2: 0.0
Nº total de bits que passam no BSC: 3476976	Nº total de bits que passam no BSC: 3476976
Nº de símbolos diferentes entre A e B: 320	Nº de símbolos diferentes entre A e B: 0
Número de caracteres:144874	
Número de bits trocados:3448	
Desired BER1: 0.001	
Actual BER1: 0.0009916663215391766	
Desired BER2: 0	
Actual BER2: 3.4512749009484104e-06	
Nº total de bits que passam no BSC: 3476976	
Nº de símbolos diferentes entre A e B: 4	

Figura 8 – Alice29.txt com código repetição (3,1)

Na figura 6, podemos observar o resultado para cada BER mencionada no enunciado por ordem decrescente. O código de repetição, como tem distância mínima de 3, consegue corrigir 1 bit errado em cada 3, com isto, o código foi capaz de corrigir 64,09% dos caracteres originalmente errados da mensagem para uma BER de 10%. Este valor é obtido a partir da seguinte fórmula:

$$\frac{\#S1 - \#S2}{\#S1} * 100$$

onde **S1** é o número de caracteres errados sem correção, **S2** é o número de caracteres errados com código de correção. **S1** e **S2** tem de ter sido obtidos a partir da mesma BER. Substituindo na fórmula temos $\frac{\#S1 - \#S2}{\#S1} * 100 = \frac{82541 - 29639}{82541} * 100 \approx 64,09\%$. Este valor é muito aproximado pois se os testes fossem executados novamente seriam obtidos outros valores.

3.3 Alínea 1(iii) – Código de Hamming (7,4)

```

./testFiles/Alice29.txt
-----
Número de caracteres:144874
Número de bits trocados:202978

Desired BER1: 0.1
Actual BER1: 0.10007612526352949

Desired BER2: 0
Actual BER2: 0.06705740850670237
Nº total de bits que passam no BSC: 2028236
Nº de símbolos diferentes entre A e B: 40171
-----
Número de caracteres:144874
Número de bits trocados:20099

Desired BER1: 0.01
Actual BER1: 0.009909596319166014

Desired BER2: 0
Actual BER2: 0.0008291687949528556
Nº total de bits que passam no BSC: 2028236
Nº de símbolos diferentes entre A e B: 569
-----
Número de caracteres:144874
Número de bits trocados:2069

Desired BER1: 0.001
Actual BER1: 0.0010200982528660372

Desired BER2: 0
Actual BER2: 4.314093626185513e-06
Nº total de bits que passam no BSC: 2028236
Nº de símbolos diferentes entre A e B: 2
-----
Número de caracteres:144874
Número de bits trocados:197

Desired BER1: 0.0001
Actual BER1: 9.712873649811955e-05

Desired BER2: 0
Actual BER2: 0.0
Nº total de bits que passam no BSC: 2028236
Nº de símbolos diferentes entre A e B: 0
-----
Número de caracteres:144874
Número de bits trocados:19

Desired BER1: 1e-05
Actual BER1: 9.367746159717115e-06

Desired BER2: 0
Actual BER2: 0.0
Nº total de bits que passam no BSC: 2028236
Nº de símbolos diferentes entre A e B: 0
-----
Número de caracteres:144874
Número de bits trocados:2069

Desired BER1: 0.001
Actual BER1: 0.0010200982528660372

Desired BER2: 0
Actual BER2: 4.314093626185513e-06
Nº total de bits que passam no BSC: 2028236
Nº de símbolos diferentes entre A e B: 2
-----

```

Figura 9 - Alice29.txt com Código Hamming (7,4)

Na figura 7, está presente o resultado experimental para o código de Hamming (7,4), um código com a distância mínima de 3, capaz de corrigir um bit errado em cada 7. Se usarmos a mesma equação da subsecção anterior temos que $\frac{\#S1-\#S2}{\#S1} * 100 = \frac{82541-40171}{82541} * 100 \approx 51,33\%$ dos caracteres originalmente errados da mensagem foram corrigidos. Como podemos observar o código de repetição (3,1) tem melhor desempenho do que o código de Hamming (7,4) para valores de BERS maiores.

Para valores de BERS inferiores o código de Hamming (7,4) tem melhor desempenho, visto que para uma BER de 0,1 % , apenas 2 caracteres da mensagem vieram errados, no código de repetição (3,1) foram 4 caracteres errados.

3.4 Alínea 1(b) – Técnica de entrelaçamento

Na realização destes testes usámos uma *string* em vez de um ficheiro. A *string* usada foi “Hello, how are you?are you good?”.

```
-----Repetição (3,1)-----
Desired BER1: 0.1
Actual BER1: 0.08723958333333333
Desired BER2: 0
Actual BER2: 0.01171875
Hello, Hov are you?are you Good?
Nº total de bits que passam no BSC: 768
Nº de símbolos diferentes entre A e B: 3

Desired BER1: 0.01
Actual BER1: 0.009114583333333334
Desired BER2: 0
Actual BER2: 0.0
Hello, how are you?are you good?
Nº total de bits que passam no BSC: 768
Nº de símbolos diferentes entre A e B: 0

Desired BER1: 0.001
Actual BER1: 0.0
Desired BER2: 0
Actual BER2: 0.0
Hello, how are you?are you good?
Nº total de bits que passam no BSC: 768
Nº de símbolos diferentes entre A e B: 0

Desired BER1: 0.0001
Actual BER1: 0.0
Desired BER2: 0
Actual BER2: 0.0
Hello, how are you?are you good?
Nº total de bits que passam no BSC: 768
Nº de símbolos diferentes entre A e B: 0
```

figura 11 - Técnica de entrelaçamento com código de repetição (3,1)

```
-----Hamming (7,4)-----
Desired BER1: 0.1
Actual BER1: 0.08705357142857142
Desired BER2: 0
Actual BER2: 0.04296875
Hello, "ho- are you?are )?u gojD?
Nº total de bits que passam no BSC: 448
Nº de símbolos diferentes entre A e B: 7

Desired BER1: 0.01
Actual BER1: 0.011160714285714286
Desired BER2: 0
Actual BER2: 0.0
Hello, how are you?are you good?
Nº total de bits que passam no BSC: 448
Nº de símbolos diferentes entre A e B: 0

Desired BER1: 0.001
Actual BER1: 0.002232142857142857
Desired BER2: 0
Actual BER2: 0.0
Hello, how are you?are you good?
Nº total de bits que passam no BSC: 448
Nº de símbolos diferentes entre A e B: 0

Desired BER1: 0.0001
Actual BER1: 0.0
Desired BER2: 0
Actual BER2: 0.0
Hello, how are you?are you good?
Nº total de bits que passam no BSC: 448
Nº de símbolos diferentes entre A e B: 0
```

figura 10 - Técnica de entrelaçamento com código de Hamming (7,4)

Na figura 8, podemos observar o que foi mencionado anteriormente, para BER's maiores o código de repetição (3,1) tem um desempenho melhor em relação ao código de Hamming (7,4). Neste caso o código de repetição apresentou apenas 3 caracteres errados de uma mensagem com 32 caracteres, noutro caso, o código de Hamming apresentou 7 caracteres errados. O Hamming apresenta mais caracteres errados, pois, se houver mais do que 2 bits na mesma palavra errados o código de Hamming, como tem distância mínima 3, não consegue corrigir mais do que um bit errado. Isto quer dizer que os 7 caracteres errados por parte do código de Hamming provavelmente têm 2 ou mais bits errados.

3.5 Alínea 2(a) – Funcionamento de SCD

Para assegurar a correta operacionalidade do sistema SCD que desenvolvemos, realizámos uma transmissão de dados utilizando uma sequência de bits que representa uma progressão geométrica. Nessa sequência, o primeiro termo foi estabelecido como sendo 2, e a razão entre os termos subsequentes foi definida como 1,5.

Essa transmissão de dados permitiu avaliar a capacidade do sistema SCD em transmitir e receber informações corretamente, mantendo a integridade dos dados ao longo do processo de comunicação. Ao verificar se a sequência recebida no recetor corresponde corretamente à sequência enviada pelo emissor, podemos concluir se o sistema SCD está funcionando adequadamente ou se há problemas que precisam ser resolvidos.

2
3
4.5
6.75
10.13
15.19
22.78
34.17
51.26
76.87
. . .

figura 12 - 10 primeiros termos da progressão geométrica

3.6 Alíneas 2(b) e 2(c) – Inserção de Padrões de erro

Como podemos observar na imagem abaixo, o valor muda bastante após aplicarmos um erro de apenas um bit e apercebemos-mos do quão importante é ter sistemas de deteção e correção de erros.

```

Índice do termo: 1
Valor do termo                               Valor do Erro                               Result
01000000000000000000000000000000 xor 0000000000000000000000000000100 -> 0100000000000000000000000000100
Before Error(float): 2.0
After error(float): 2.0000009536743164
Checksum verification: Passed
-----
Índice do termo: 2
Valor do termo                               Valor do Erro                               Result
01000000010000000000000000000000 xor 0000000000000000010000000000000 -> 01000000010000000000010000000000
Before Error(float): 3.0
After error(float): 3.00048828125
Checksum verification: Passed
-----
Índice do termo: 3
Valor do termo                               Valor do Erro                               Result
01000000100100000000000000000000 xor 0000000000000000000100000000000 -> 010000001001000000000001000000000
Before Error(float): 4.5
After error(float): 4.500244140625
Checksum verification: Passed

```

figura 13 - Resultados de aplicações de erros sobre progressão geométrica

4. Conclusões

Neste relatório, foram abordados dois exercícios relacionados a sistemas de comunicação digital. No primeiro exercício, foram considerados o modelo de canal binário simétrico (BSC) e a aplicação de técnicas de correção de erros, como o código de repetição e o código de Hamming. Foram calculados os valores de taxa de erro de bit (BER) para diferentes probabilidades de erro de bit (p) e comparados os resultados obtidos.

No primeiro caso, sem a aplicação de qualquer técnica de correção de erros, o BER1 correspondeu à taxa de erro de bit na transmissão direta pelo BSC. Em seguida, ao aplicar o código de repetição (3, 1), o BER2 diminuiu significativamente, demonstrando a eficiência dessa técnica na correção de erros. Por fim, com a aplicação do código de Hamming (7, 4), o BER3 apresentou uma melhoria adicional na taxa de erro de bit.

Além disso, foram analisados o número total de bits que passaram pelo BSC e o número de símbolos diferentes entre os arquivos originais e os arquivos resultantes. Essas métricas forneceram insights sobre a integridade dos dados transmitidos.

No segundo exercício, foi abordada a implementação de um sistema de comunicação simplex entre um Arduino (emissor) e um PC (recetor) utilizando a plataforma Arduino e a linguagem Python. Foi implementado um sistema de comunicação de dados que transmitia os primeiros N termos de uma progressão geométrica. Além disso, foi aplicada a técnica de detecção de erros usando o checksum de Fletcher.

Foram realizados testes experimentais para verificar o correto funcionamento do sistema de comunicação e, posteriormente, foram introduzidos padrões de erro isolados e “burst” para avaliar a capacidade de detecção de erros do checksum de Fletcher. Os resultados obtidos demonstraram a eficiência dessa técnica na detecção de erros, tanto em bits isolados quanto em “burst”.

Em suma, os exercícios abordaram conceitos fundamentais de sistemas de comunicação digital, como a transmissão por canais com erro e a aplicação de técnicas de correção e detecção de erros. Os resultados obtidos evidenciaram a importância dessas técnicas na melhoria da integridade e confiabilidade dos dados transmitidos, bem como a capacidade do checksum de Fletcher na detecção de erros.